

האוניברסיטה העברית

בי"ס להנדסה ולמדעי המחשב

בחינה בקורסי מבוא למדעי המחשב:

67101 – מבוא למדעי המחשב

67108 – מבוא למדעי המחשב לתלמידי תלפיות

תאריך: 20.1.2014

משך הבחינה: שלוש שעות

מועד א' - סמסטר א' – תשע"ד – 2013-2014

פרופ' יובל רבני, ד"ר אריה שלזינגר

הנחיות:

1. חומר עזר מותר - כל חומר על גבי נייר (ספרים, מחברות, דפים וכו'). אסור - מכשיר אלקטרוני כלשהו.
2. יש לענות על 3 מתוך 4 השאלות הבאות (3 בלבד). משקל כל השאלות זהה (34 נק'), אך הציון המקסימלי הוא 100.
3. יש לכתוב פתרון לכל אחת מהשאלות שבחרת במחברת נפרדת. יש לציין על כריכת המחברת לאיזה שאלה המחברת מתייחסת, באופן קריא וברור. אין לענות על גבי טופס הבחינה - הוא לא נמסר בסיום הבחינה ולא יבדק.
4. יש לציין את מס' תעודת הזהות ומספר המחברת על כל מחברת.
5. הקפידו הקפדה יתרה על סדר וכתב יד קריא. בפיית'ון ההזחה קריטית להבנת הקוד. בחינה בלתי קריאה לא תזכה את כותבה במלוא הנקודות.
6. יש למחוק טיטות באופן ברור. הבדיקה לא תתייחס לטיטות כלל.
7. יש להשאיר שוליים.
8. אסור להשתמש בעט אדום.
9. הקפידו על סגנון תכנותי נאות: כתיבה באופן מודולרי, ללא שיכפול קוד וכו'.
10. תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית), אין צורך לכתוב docstring.
11. פתרון מיטבי לשאלה יחשב פתרון פשוט ויעיל ככל האפשר. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה (גם אם היא נכונה).
12. אין להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה (אלא אם כן צוין אחרת בשאלה). עם זאת, מותר להשתמש במבני נתונים נוספים אשר גודלם ומספרם אינו תלוי בגודל הקלט (למשל מותר להשתמש ברשימה בת שלשה תאים פעם אחת).
13. בכל השאלות ניתן להניח שהקלט חוקי ומקיים את תנאי השאלה. ניתן להניח כי מבני הנתונים מכילים רק ערכים תקינים ואינם **None**, אלא אם מובהר בשאלה אחרת.
14. בפתרון השאלה הינכם רשאים לבצע שימוש בשיטות שכבר מימשתם בסעיפים אחרים של אותה השאלה (אך לא בשיטות משאלות אחרות!).

בהצלחה!

הנחות בנוגע לסיבוכיות של פעולות על רשימה (list) של פיית'ון

סיבוכיות זמן ריצה	פעולה על רשימה L באורך N
$O(1)$	קבלת אורך רשימה – <code>len(L)</code>
$O(1)$	גישה לאיבר לפי אינדקס – <code>L[i]</code>
$O(1)$	הוספת איבר בסוף הרשימה – <code>L.append(data)</code>
$O(N)$	הוספת איבר במקום מוגדר ברשימה – <code>L.insert(i, data)</code>
$O(1)$	הוצאת איבר מסוף הרשימה – <code>L.pop()</code>
$O(N)$	הוצאת איבר ממקום מוגדר ברשימה – <code>L.pop(i)</code>

הנחות אלה אינן בהכרח נכונות לכל מימוש של רשימה (list) בפיית'ון, אבל לצורך ניתוח סיבוכיות במבחן יש להשתמש בהן.

שאלה 1 (34 נקודות)

א. (2 נק') מה יודפס לאחר הרצת קטע הקוד הבא:

```
x='abcdef'
y='' # empty string
k=2
print(x[0:k]+x[k+1:] , y+x[k])
```

סעיפים ב' וג' מתייחסים לפונקציה הבאה:

```
def f1(x, y=''): # y's default value is the empty string
    if len(x) == 0:
        print (y)
    else:
        for k in range(len(x)):
            f1(x[0:k] + x[k+1:], y+x[k])
```

ב. (4 נק') ציירו את עץ הקריאות הרקורסיבי לפונקציה `f1('ab')` - בדומה לעץ הקריאות הרקורסיבי לפונקציה `hanoi` אשר ראיתם בתרגול 5 - השורש יהיה `f1(x='ab', y='')` בניו יהיו כל הקריאות הנוספות שנעשו לפונקציה `f1` ממנו. יש לציין את הארגומנטים איתם נקראו הפונקציות. עלים בעץ יהיו קריאות לפונקציה שלא קוראות בעצמה ל `f1`.

ג. (4 נק') מה תדפיס הפונקציה `f1` עבור `f1('abc')` ?

ד. (2 נק') מה תדפיס הפונקציה `f1` עבור `f1('abc', 'd')` ?

ה. (4 נק') האם הרצת `f1([1,2,3])` תצליח? אם כן מה יהיה הפלט, אם לא מה הסיבה?

ו. (סה"כ 9 נק') לפניכם שלוש פונקציות:

```
def first_gen(n):
    for index in range(n):
        yield 1 if index%2 else 2

def second_gen(n):
    for index in range(n):
        yield sum(first_gen(index) )

def third_gen(n):
    if n:
        yield n + next( third_gen(n-1))
    else:
        yield 0
```

מה יודפס בכל אחד מארבעת הביטויים הבאים ?

1. (2 נק') `print(list( first_gen(5) ))`
2. (1 נק') `print(sum( first_gen(5) ))`
3. (3 נק') `print(list( second_gen(5) ))`
4. (3 נק') `print([ num for num in third_gen(4) ])`

ז. (2 נק') מה מניחה הפונקציה הבאה לגבי הקלט שלה

```
def fourth_gen(my_list):  
    first = my_list[0]  
    for value in my_list[1:] :  
        second = value  
        yield first+second  
        first = second
```

ח. (2 נק') מה עושה הפונקציה `fourth_gen(my_list)`?

ט. (2 נק') קטע הקוד הבא משתמש בפונקציה מהסעיף הקודם, אך נסיון להריץ אותו יסתיים בשגיאה. מה השגיאה וכיצד יש לתקנה (דרוש תיקון של שינוי שורת קוד אחת בלבד)?

```
my_list = [5, 4, 3, 9, 6]  
my_gen = fourth_gen(my_list)  
first = my_gen[0]  
for num in my_list:  
    if first == num:  
        print("Found")  
        break  
else:  
    print("Not Found")
```

י. (3 נק') לאחר התיקון, מה יודפס לאחר שנריץ את הקוד?

שאלה 2 (34 נקודות)

נתונה המחלקה הבאה, המשמשת לייצוג שאלות במבחן:

```
class ExamQuestion:
    def __init__(self, id, difficulty=0, duration=0):
        self.difficulty = difficulty
        self.duration = duration
        self.id = id
    def __repr__(self):
        return "q"+str(self.id)
```

כמו כן נתון הקוד הבא:

```
def my_key1(exam_question):
    return exam_question.difficulty
```

```
q1 = ExamQuestion(1,2,7)
q2 = ExamQuestion(2,5,4)
q3 = ExamQuestion(3,4,1)
exam_questions = [q1,q2,q3]
```

א. (4 נק') מה ידפיס קטע הקוד הבא:

```
print(sorted(exam_questions, key=my_key1))
```

תזכורת: הפונקציה `sorted` ממיינת את הרשימה הנתונה בסדר עולה בעזרת השוואה של ערך המפתח (`key`) של איברים שונים ברשימה.

ב. (4 נק') מה ידפיס קטע הקוד הבא:

```
print(sorted(exam_questions, key=lambda n:n.duration))
```

ג. (6 נק') כתבו פונקציית מפתח בשם `my_key3`, שבעזרתה ניתן יהיה למיין שאלות למבחן כך שהמשקל של קושי השאלה (`difficulty`) הוא כפול מהמשקל של אורך השאלה (`duration`). לדוגמא, שאלה ברמת קושי 3 ובאורך 1 תמויין לפני שאלה ברמת קושי 2 ובאורך 4 אבל אחרי שאלה ברמת קושי 2 ובאורך 2.

ד. (7 נק') השלימו את קטע הקוד הבא כך שבהינתן שתי רשימות ממויינות ופונקציית המפתח שבעזרתה הרשימות מויינות, הפונקציה מחזירה רשימה חדשה וממויינת (ע"פ פונקציית המפתח) המכילה את אברי שתי הרשימות. במקרה של שיוויון בין אברים משתי הרשימות על האיברים מהרשימה הראשונה להופיע ראשונים ברשימה החדשה. יש להשלים (לא על גבי הטופס) שלוש שורות קוד (מסומנות בקו תחתון), ציינו את מספר השורה שאתם משלימים.

```
def merge(L1, L2, key):
    idx1, idx2 = 0, 0
    res=[]
    (1) while ____:
    (2)     if ____:
        res.append(L1[idx1])
        idx1 += 1
    else:
        res.append(L2[idx2])
        idx2 += 1
    (3) return ____
```

ה. (2 נק') בהינתן שאורך הרשימה הראשונה הוא n ואורך הרשימה השנייה הוא m , וכן שזמן הריצה של פונקציית המפתח הוא $O(1)$, מה זמן הריצה של הפונקציה `merge` ?

ו. (8 נק') צוות מבוא למדעי הדשא הוא צוות גדול וכולם משתתפים בכתיבת השאלות למבחן, כל אחד מחברי הצוות כתב מספר שאלות רב, מיין אותן על פי פונקציית המפתח שנבחרה והעביר אותן לפרופסור דוד. כעת דוד מעוניין להכין רשימה אחת ממויינת (ע"פ פונקציית המפתח) שבה כל השאלות.
ממשו את הפונקציה:

```
def sort_sorted_exam_questions_lists(questions, key):
```

הפונקציה מקבלת רשימה של רשימות של שאלות ממויינות ומחזירה רשימה חדשה וממויינת של כל השאלות.

הערה: לא ניתן להשתמש בפונקציה `sorted` של פיית'ון אך ניתן להשתמש בפונקציה `merge` מסעיף ד'.

ז. (3 נק') מה סיבוכיות זמן הריצה והזיכרון של סעיף ו' כתלות בסה"כ מספר השאלות? לצורך הניתוח הניחו שכל אחד מ k המתרגלים כתב n שאלות. הסבירו כיצד הגעתם לפתרון שלכם.

שאלה 3 (34 נקודות)

נתונה המחלקה `Node` המייצגת איבר ברשימה המקושרת:

```
class Node:
    def __init__(self, data, next=None):
        self.data = data
        self.next = next
```

ונתונה המחלקה הבאה המייצגת רשימה מקושרת ממויינת:

```
class SortedLinkedList:
    def __init__(self):
        self.head = None # Points to the smallest Node
        self.nodes_list = []
```

הרשימה המקושרת ממויינת בסדר עולה, כלומר כל איבר הוא קטן או שווה מהאיבר הבא (`next`) או במילים אחרות, לכל `node` ברשימה מתקיים `node.data <= node.next.data`. אחד החסרונות ברשימה מקושרת, כפי שראינו, הוא שזמן הגישה לאיבר ברשימה הוא $O(n)$. בכדי לאפשר גישה יעילה יותר, מחלקה זו שומרת גם רשימה (`list`) של איברי הרשימה המקושרת (ה `Nodes`), אך רשימה זו אינה בהכרח ממויינת. שמירת רשימת התאים מאפשרת, באמצעות האלגוריתם שנראה בהמשך השאלה, גישה לכל איבר בזמן ממוצע טוב מ $O(n)$.

א. (5 נק') ממשו את השיטה (method) הבאה של המחלקה `SortedLinkedList`:

```
def sort_list(self):
```

השיטה ממיינת את ה `self.nodes_list` בסדר עולה. יש לממש שיטה זו בסיבוכיות זמן ריצה של $O(n)$. אין להשתמש בפונקציות המיון של פיית'ון. רמז: זכרו שהרשימה המקושרת ממויינת.

ב. (12 נק') ממשו את השיטה הבאה של המחלקה `SortedLinkedList`:

```
def find_before(self, max_data):
```

השיטה מחזירה את האיבר (`Node`) שהערך שלו (ה `data` שלו) הוא הגדול ביותר מבין האיברים שערכיהם קטנים מ `max_data` הנתון כקלט לשיטה.

על מנת למצוא את האיבר בצורה יעילה השיטה תפעל על פי האלגוריתם הבא:

נסמן ב N את מספר האיברים ברשימה. השיטה תבצע לכל היותר שורש N פעמים (ניתן להשתמש ב `math.sqrt(N)`) את רצף הפעולות הבא:

1. תגדיל אינדקס i בין 0 לגודל הרשימה. לצורך כך ניתן להשתמש בפונקציה `random.randint(a,b)`

המחזירה מספר שלם בין a ל b (כולל).

2. תשווה את ערך האיבר הנמצא באינדקס i ברשימה `nodes_list` ל `max_data`.

3. אם ערכו גדול או שווה לו, או שערכו קטן מהערך המקסימלי שנמצא עד שלב זה תחזור לשלב 1.

4. אחרת (ערכו קטן מ `max_data` וגדול מהערך המקסימלי שנמצא עד כה), השיטה תבדוק את ערכו של האיבר

הבא ברשימה המקושרת: באם ערכו של האיבר הבא גדול יותר מ `max_data` (או שאין איבר הבא) תחזיר את ה

`Node` הנמצא באינדקס i , אחרת תעדכן את אינדקס האיבר הגדול ביותר שנמצא עד כה להיות i ותמשיך.

לאחר שורש N פעמים, השיטה תתקדם איבר איבר בסדר עולה החל מהאיבר הגדול ביותר שנמצא עד כה (ושערכו

קטן מ `max_data`) עד שימצא איבר שיש להחזיר.

אם `max_data` קטן מערכו של האיבר הקטן ביותר, או אם הרשימה ריקה, על השיטה להחזיר `None`.

ג. (11 נק') ממשו את השיטה הבאה של המחלקה `SortedList`:

```
def insert(self, data):
```

השיטה מוסיפה איבר חדש לרשימה המקושרת עם הערך הנתון למקום המתאים ברשימה הממוינת. בנוסף, על השיטה להוסיף את האיבר החדש לסוף ה `list_nodes`.

ד. (3 נק') מה סיבוכיות זמן הריצה של שיטת ההכנסה במקרה הגרוע? השוו את סיבוכיות זמן הריצה לסיבוכיות זמן ריצת הכנסה של איבר לרשימה מקושרת במקרה הרגיל (ללא רשימת העזר `list_nodes`).

ה. (3 נק') הדגימו ריצה שבה מקבלים את הסיבוכיות הגרועה ביותר של השיטה `insert` שמימשותם עבור רשימה בגודל 4. פרטו את המספרים האקראיים שהוגרלו.

שאלה 4 (34 נקודות)

א. (6 נק') נתונה פונקציית הגיבוב (hash function) הבאה:

$$h(k) = k \% 7$$

ציירו את טבלת הגיבוב (hash table) שתיווצר לאחר הכנסת האיברים הבאים בעזרת $h(k)$ (משמאל לימין):

19, 26, 13, 48, 17

בכל אחד מהתרחישים הבאים:

1. התנגשויות מטופלות ע"י שרשור (chaining)
2. התנגשויות מטופלות ע"י גישוש ליניארי (linear probing) - כלומר $h(k,i) = h(k) + i$ כאשר i גדל באחד בכל פעם שהתא תפוס.

ב. (8 נק') פרופסור שלזני מציעה שיטה לטיפול בהתנגשויות ע"י שרשור (chaining). בכל תא בטבלת הגיבוב נחזיק רשימה

של פיית'ון (list) ממיינת של הערכים שמופו לתא זה. השוו את סיבוכיות הזמן של הפעולות הבאות לפי שיטתה של

פרופסור שלזני, לעומת הסיבוכיות של אותן פעולות כאשר הרשימות בכל תא אינן ממיינות. הפעולות שיש לנתח הינן:

1. חיפוש מוצלח (בהינתן מפתח)
2. חיפוש לא מוצלח (בהינתן מפתח)
3. הכנסת איבר חדש
4. מחיקת איבר (בהינתן המפתח שלו)

עבור הסעיפים הבאים, נתונה המחלקה **HashTable** ופונקציות הממומשות בה:

```
class HashTable:
```

```
    # מאתחלת טבלת גיבוב. כל הפעולות בטבלת הגיבוב יעשו על ידי פונקציית הגיבוב hash שהתקבלה כקלט.
    __init__(self, hash)

    # מכניסה את obj לתוך טבלת הגיבוב.
    insert(self, obj)

    # מחזירה אמת אם obj נמצא בתוך טבלת הגיבוב.
    search(self, obj)
```

בסעיפים הבאים יש להשתמש אך ורק בפונקציות של **HashTable** (ולא באף פונקציית ספרייה של פיית'ון). בנוסף ניתן להניח כי **hash** הינה פונקציית גיבוב מוצלחת (מעט התנגשויות ומבוצעת ב $O(1)$), וכי הערכים בכל אחת מהרשימות (שינתנו כקלט) ייחודיים (אף ערך לא חוזר פעמיים).

ג. (6 נק') ממשו את הפונקציה הבאה:

```
def is_subset(S,T,hash)
```

כאשר S ו- T הינן רשימות (`list`, לא רשימות מקושרות). הפונקציה מחזירה אמת אם ורק אם S מוכלת בתוך רשימת T . סיבוכיות הזמן של הפונקציה צריכה להיות $O(|T|)$ במקרה הממוצע ($|T|$ = מספר האיברים ברשימת T). הסבירו מדוע המימוש הוא בסיבוכיות זו.

ד. (6 נק') ממשו את הפונקציה הבאה:

```
def xor(S,T,hash)
```

כאשר S ו- T הינן רשימות (`list`, לא רשימות מקושרות). הפונקציה מחזירה רשימה חדשה שהיא "או-מוציא" (`XOR`) של שתי הרשימות S ו- T . כלומר, ברשימה החדשה יכללו רק איברים המופיעים באחת הרשימות אבל לא בשניהן. אם אין איברים כאלה תוחזר רשימה ריקה. סיבוכיות הזמן של הפונקציה צריכה להיות $O(|T|+|S|)$ במקרה הממוצע. הסבירו מדוע המימוש הוא בסיבוכיות זו.

ה. (8 נק') נתונה המחלקה הבאה:

```
class Node:
```

```
    def __init__(self,value,next):
```

```
        self.value = value
```

```
        self.next = next
```

ממשו את הפונקציה הבאה:

```
def is_equal(s,t,hash)
```

כאשר s ו- t הינם מצביעים לראש של רשימות מקושרות (`Node`). הפונקציה מחזירה אמת אם הערכים ברשימות המקושרות המתחילות ב- s ו- t הינם זהים (לא חייבים להיות באותו סדר). סיבוכיות הזמן של הפונקציה צריכה להיות $O(\min\{|S|,|T|\})$ במקרה הממוצע, כאשר $|S|$ = מספר האיברים ברשימה המתחילה ב- s , ו- $|T|$ = מספר האיברים ברשימה המתחילה ב- t . הסבירו מדוע המימוש הוא בסיבוכיות זו.